

Real-Time Ride Booking & Fleet Management System

RideFlow — Flutter Mobile Application with Node.js Backend

Project No.	11
Group	Group 4
Category	Transportation · Real-time · Geolocation

Team Members

Vamshi Shetty

Rayan Rawat

Tanishq Chopra

Yash Parab

ITM Skills University, Kharghar
Flutter Application Development — Project Assignment

Table of Contents

- 1. Project Overview
- 2. Technology Stack
- 3. System Architecture
- 4. Flutter Client Architecture
- 5. Backend Architecture
- 6. Authentication & Session Flow
- 7. Ride Lifecycle Flow
- 8. REST API Reference
- 9. Data Model
- 10. Fare Calculation Logic
- 11. Seeded Demo Accounts
- 12. Theming & UI Design System
- 13. Implementation Scope

1. Project Overview

RideFlow is a ride-booking platform where riders can request rides and drivers can manage their availability and accept trips. The system is built as a cross-platform Flutter application (**project name: rideflow**) backed by a lightweight Node.js/Express REST API. The backend handles user authentication, ride lifecycle management, fare calculation, and trip history, with data persisted to a local JSON file store.

The application is designed around two user roles — **riders**, who request trips between a pickup location and a destination, and **drivers**, who register their vehicle details and respond to incoming ride requests. Both roles share a single authentication screen, with the interface adapting its fields and copy based on the selected role.

Key Focus Areas

- **User & driver authentication** — unified sign up / sign in flow with role-based fields.
- **Ride lifecycle management** — requests move through pending, accepted, and rejected states.
- **Fare calculation logic** — distance-based pricing with a driver-availability surge multiplier.
- **Trip history** — per-rider ride records retrievable through the API.

Project Tags

Transportation · Real-time · Geolocation

2. Technology Stack

Layer	Technology
Frontend framework	Flutter (Dart SDK ^3.13.1), Material 3 design system
Local session storage	shared_preferences ^2.3.2 — persists login state and role on-device
Backend runtime	Node.js with Express.js ^4.21.2
Cross-origin support	cors ^2.8.5 middleware, enabled globally
Data persistence	Flat-file JSON database at backend/data/db.json
ID generation	Node.js built-in crypto.randomUUID()
Supported client platforms	Android, iOS, Web, Windows, macOS, Linux (full Flutter multi-platform scaffold present)
Dev tooling	flutter_lints ^6.0.0 (static analysis), nodemon ^3.1.7 (backend hot-reload), devtools_options.yaml
Build system	Standard Flutter build pipeline (build/ directory with web, native_assets and test caches)

3. System Architecture

RideFlow follows a classic client–server split: the Flutter application renders the UI and manages local session state, while the Express backend exposes a stateless REST API layered over a single JSON document acting as the datastore. The diagram below maps each client screen and backend route group to the data they read or write.

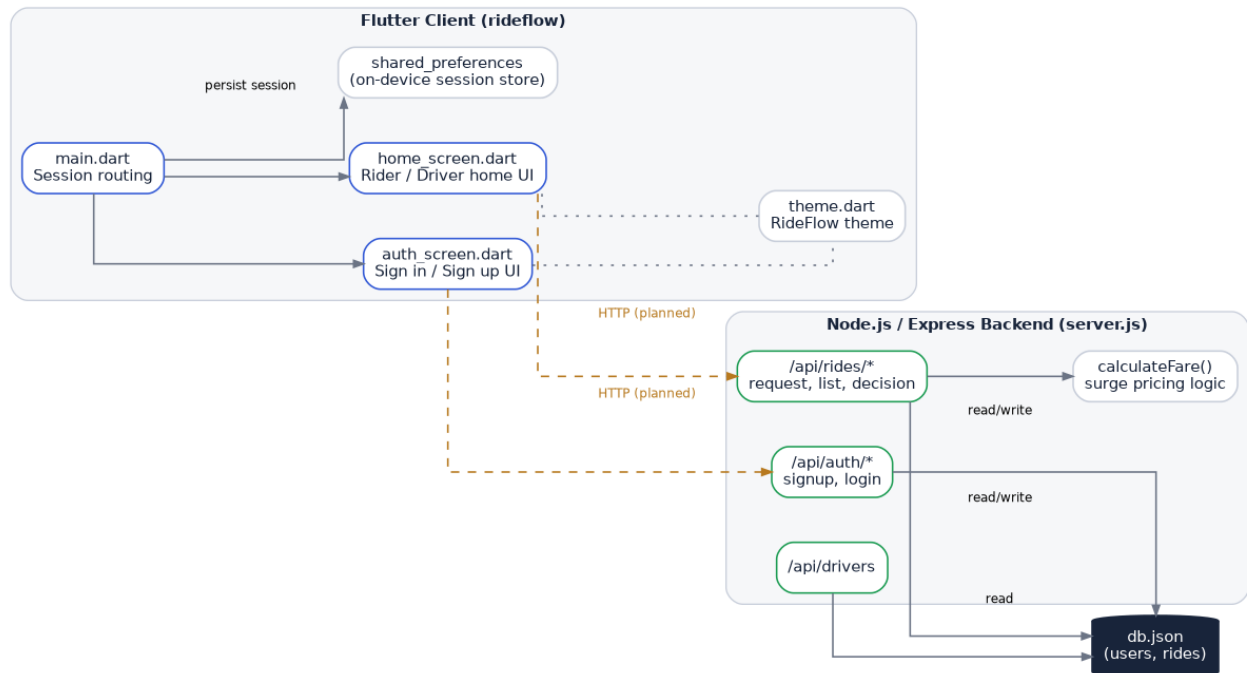


Figure 1 — RideFlow system architecture: Flutter client, Express API, and JSON datastore.

The client and server currently run as independently deployable units — the Flutter app can be launched on any supported platform while `server.js` runs standalone via `node server.js` or `npm run dev` (nodemon) on port 3000 by default (configurable via the `PORT` environment variable).

4. Flutter Client Architecture

The Flutter client follows a simple, stateful root-widget architecture rather than a dedicated state-management package (no `Provider`, `Riverpod`, or `Bloc` dependency is declared in `pubspec.yaml`). Session state (login flag and rider/driver role) is restored from on-device storage at startup, and the root widget swaps between the authentication flow and the home flow accordingly.

File	Lines	Responsibility
lib/main.dart	92	App entry point. <code>RideFlowApp</code> (<code>StatefulWidget</code>) restores session via <code>shared_preferences</code> on launch (<code>_restoreSession</code>), exposes <code>_openHome()</code> to persist login state after auth, and <code>_signOut()</code> to clear it. A <code>GlobalKey<NavigatorState></code> drives programmatic navigation to <code>HomeScreen</code> after login.
lib/auth_screen.dart	219	Combined sign-in / sign-up screen with a segmented Sign in / Create account toggle and a User / Driver role selector. Sign-up collects name, email, password, and (for drivers) license number, car registration number and car brand. Includes one-tap 'User demo' / 'Driver demo' buttons that prefill the demo credentials seeded in the backend, plus a 'Continue with demo account' shortcut.

File	Lines	Responsibility
lib/theme.dart	33	Centralised ThemeData (buildRideFlowTheme) defining the app's color palette (ink, muted, blue), Material 3 color scheme, input field styling, and filled-button styling used across screens.
lib/home_screen.dart	0	Declared and imported by main.dart as the post-login destination, constructed as HomeScreen(isDriver: ..., onSignOut: ...). The file exists in the project structure as the designated location for the rider and driver home UI.

Widget & State Summary

State field	Type	Owner widget	Purpose
_isLoggedIn	bool	RideFlowApp	Whether a session is currently active
_isDriver	bool	RideFlowApp	Which role the active session belongs to
_isReady	bool	RideFlowApp	Guards the UI until shared_preferences has been read
_isSignUp	bool	AuthScreen	Toggles between Sign in and Create account modes
_isDriver	bool	AuthScreen	Toggles between User and Driver role fields
_obscurePassword	bool	AuthScreen	Controls password field visibility

Form Fields & Validation Rules

Field	Shown in	Validation rule
Full name	Sign up (all roles)	Required, non-empty after trim
Email address	Sign in / Sign up	Must contain '@'
Password	Sign in / Sign up	Minimum 6 characters
Driving license number	Sign up (driver only)	Minimum 6 characters after trim
Car registration number	Sign up (driver only)	Minimum 6 characters after trim
Car brand and model	Sign up (driver only)	Required, non-empty after trim

6. Authentication & Session Flow

Session handling is centralised in RideFlowApp. On every cold start the app attempts to restore a previous session before deciding which screen to show, which avoids forcing an already-authenticated user to sign in again. Both the sign-in and sign-up paths converge on the same `_openHome()` call once form validation succeeds.

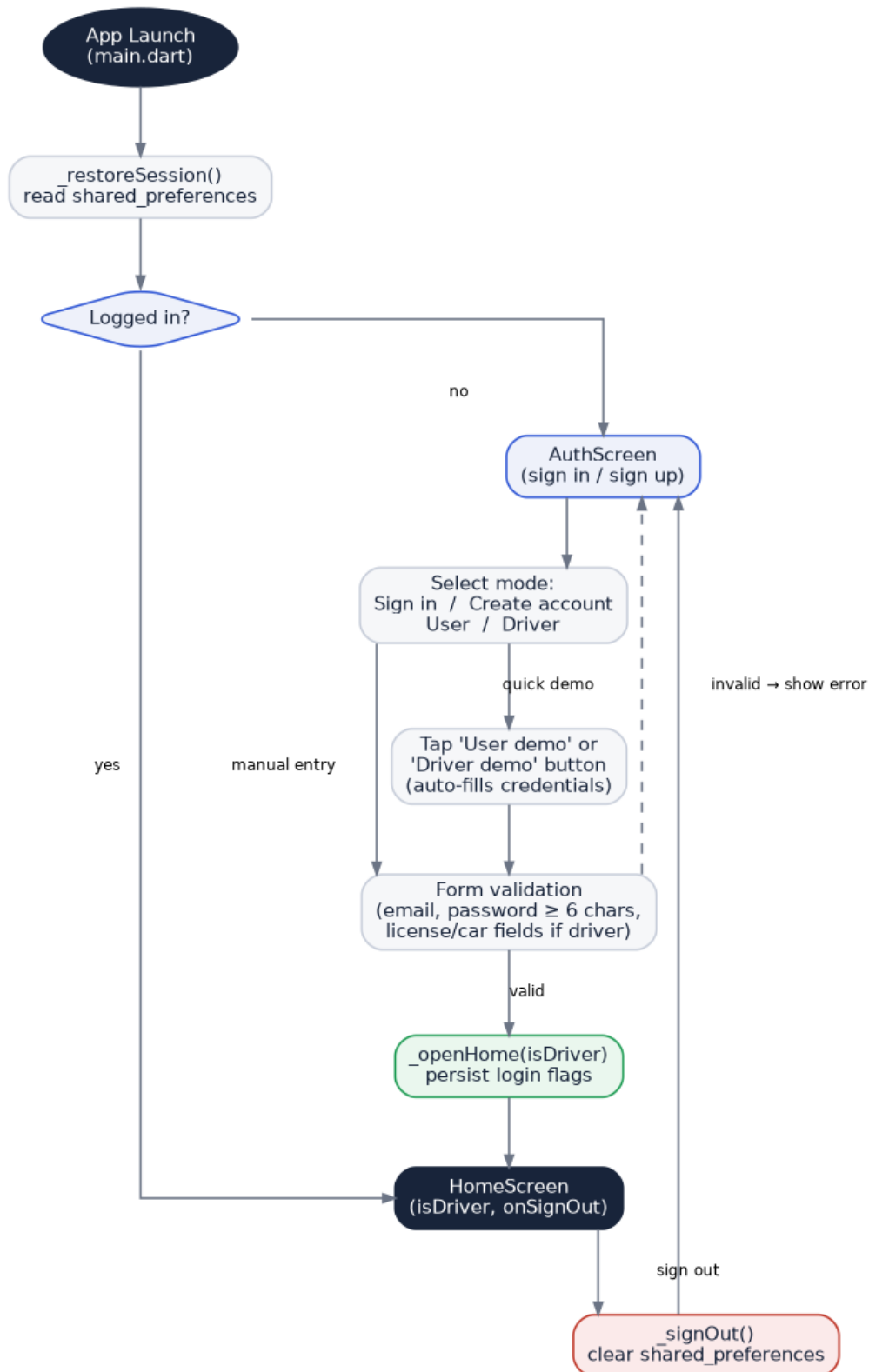


Figure 2 — App launch, authentication and sign-out flow.

Persisted Keys (shared_preferences)

Key	Type	Behaviour
rideflow_user_logged_in	bool	Set true on successful auth; cleared on sign out
rideflow_user_is_driver	bool	Set to the selected role on successful auth

7. Ride Lifecycle Flow

A ride begins when a rider submits pickup and destination details. The backend validates the request, computes the fare using the current driver pool size, and stores the ride with a 'pending' status. A driver's accept/reject decision then transitions the ride to its terminal state for that attempt.

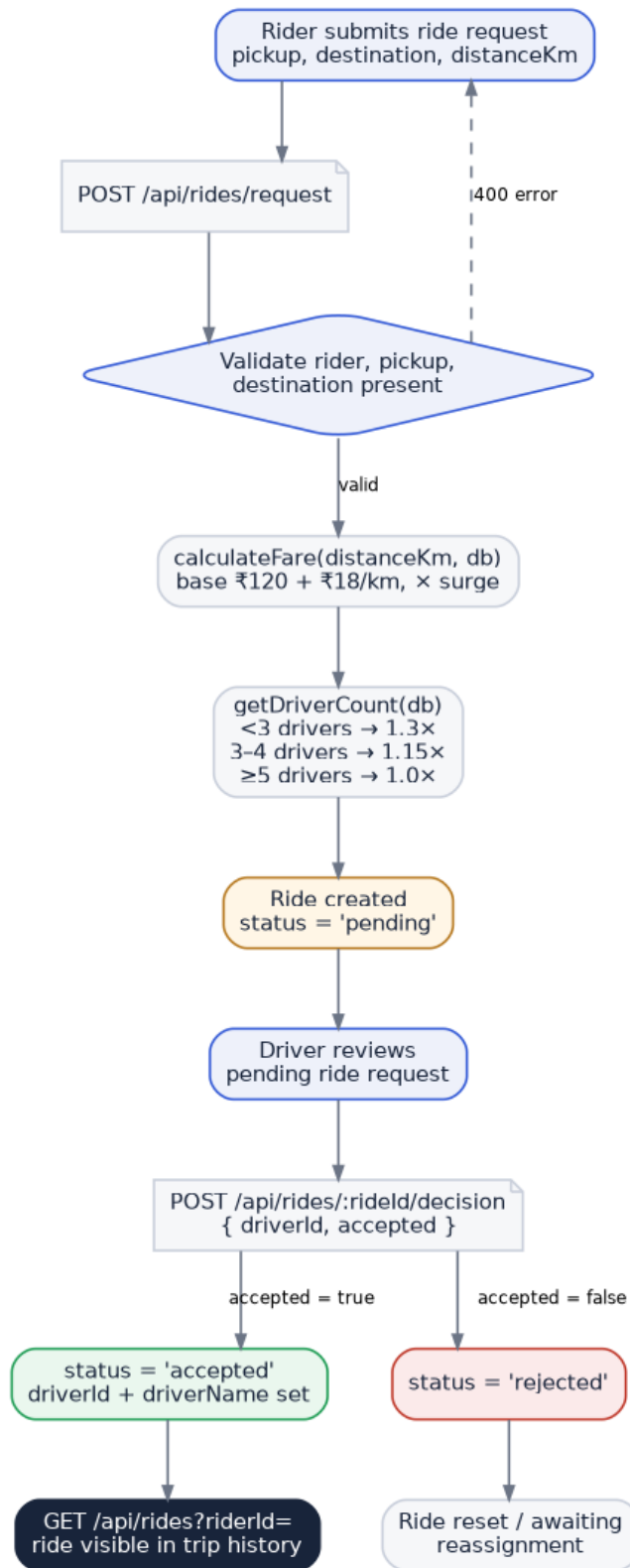


Figure 3 — Ride request, fare calculation, and driver decision flow.

8. REST API Reference

All endpoints are mounted on the Express app root and communicate using JSON. There is no authentication middleware protecting routes — the demo token returned from signup/login is not currently verified on subsequent requests.

Method	Endpoint	Body / Params	Description
GET	/api/health	—	Health check — returns { ok, message, timestamp }.
POST	/api/auth/signup	name, email, password, role, licenseNumber*, carNumber*, carBrand*	Registers a new rider or driver. Rejects duplicate emails (409). Returns the sanitized user and a demo token (201).
POST	/api/auth/login	email, password	Authenticates by email + password (case-insensitive email match). Returns user + token, or 401 on mismatch.
GET	/api/drivers	—	Lists all registered drivers with passwords stripped from the response.
POST	/api/rides/request	riderId, pickupLocation, destination, distanceKm	Creates a ride request. Computes fare via calculateFare(). New ride status starts as 'pending' (201).
GET	/api/rides	riderId (optional query param)	Lists all rides, or filters to one rider's trip history.
POST	/api/rides/:rideId/decision	driverId, accepted (bool)	Driver accepts or rejects a ride; sets status to 'accepted' or 'rejected' and records driverId/driverName.

* required only when role = 'driver'

Response Status Codes

Code	Meaning	When returned
200	OK	Successful GET, login, or ride decision
201	Created	Successful signup or ride request
400	Bad Request	Missing required fields
401	Unauthorized	Invalid email/password combination on login
404	Not Found	Rider, driver, or ride ID does not exist
409	Conflict	Signup email already registered

9. Data Model

The datastore is a single JSON document (backend/data/db.json) with two top-level collections, users and rides. `ensureDb()` creates the file with seeded demo data on first run if it does not already exist; `readDb()` / `writeDb()` perform full-document reads and writes for every request.

User

Field	Type	Notes
id	string (UUID)	Unique user identifier
name	string	Full name
email	string	Login identifier (lowercased, unique)
password	string	Stored as plain text in this demo datastore
role	'rider' 'driver'	Account type
licenseNumber, carNumber, carBrand	string	Populated for driver accounts only
createdAt	ISO timestamp	Account creation time

Ride

Field	Type	Notes
id	string (UUID)	Unique ride identifier
riderId / driverId	string	References to the requesting rider and assigned driver
pickupLocation, destination	string	Free-text locations entered at request time
distanceKm	number	Trip distance; defaults to 12 km if omitted/invalid
fare	number	Computed via the fare calculation formula at request time
status	'pending' 'accepted' 'rejected'	Ride lifecycle state
driverName	string	Set once a driver makes a decision
createdAt / updatedAt	ISO timestamp	Audit timestamps

10. Fare Calculation Logic

Fare = (base fare + distance in km × per-km rate) × surge multiplier, where the surge multiplier is derived from the number of currently registered driver accounts (`getDriverCount`) — fewer available drivers produce a higher surge, modelling basic supply-and-demand pricing.

Parameter	Value
Base fare	■120 flat, added regardless of distance

Parameter	Value
Per-km rate	■18 / km, multiplied by distanceKm
Surge: fewer than 3 drivers	1.3× applied to (base + distance cost)
Surge: 3–4 drivers	1.15× applied to (base + distance cost)
Surge: 5 or more drivers	1.0× — no surge
Rounding	Final fare rounded to the nearest whole rupee (Math.round)

Worked example: a 10 km trip with 2 registered drivers → $(120 + 10 \times 18) \times 1.3 = 300 \times 1.3 = \text{■}390$. The same trip with 6 registered drivers → $(120 + 180) \times 1.0 = \text{■}300$.

11. Seeded Demo Accounts

The backend seeds three demo accounts on first run (all with password 123456), matching the 'User demo' / 'Driver demo' quick-fill buttons on the sign-in screen. This allows the app to be demonstrated end-to-end without registering new accounts.

Name	Email	Role	Vehicle / License
Alex Morgan	user@rideflow.com	Rider	—
Raj Sharma	driver@rideflow.com	Driver	Maruti Suzuki Dzire · MH 43 BK 7821 · MH1420230012345
Amit Verma	amit@rideflow.com	Driver	Hyundai Creta · MH 02 AB 4411 · MH1320230054321

12. Theming & UI Design System

`buildRideFlowTheme()` in `theme.dart` centralises the app's visual language so every screen shares a consistent look. The theme is built on Material 3 (`useMaterial3: true`) with a seeded color scheme derived from the brand blue.

Token	Value	Usage
ink	#18243A	Primary text color
muted	#6B7485	Secondary / helper text color
blue	#3259D6	Brand / primary action color, seeds the ColorScheme
Scaffold background	#F6F7F9	App-wide background
Font family	Arial	Default text theme font
Input fields	15px rounded corners, #E1E5EC border, blue 1.5px focus border	InputDecorationTheme
Filled buttons	Blue background, white text, 15px rounded corners	FilledButtonThemeData

13. Implementation Scope

The table below summarises what each layer of the codebase currently provides, based directly on the submitted source.

Area	Status
Project scaffold	Full Flutter multi-platform scaffold: Android, iOS, Web, Windows, macOS and Linux build targets are all present and configured.
Authentication UI	Complete sign-in / sign-up screen (<code>auth_screen.dart</code>) with role switching, field validation, and demo account shortcuts.
Session persistence	Login state and role are saved and restored on-device via <code>shared_preferences</code> (<code>main.dart</code>).

Area	Status
Design system	A single shared theme (theme.dart) defines the color palette and component styling used across the app.
Backend API	Express server (server.js) implementing signup, login, driver listing, ride request, ride listing, and ride accept/reject endpoints.
Fare engine	Distance-based fare formula with driver-availability surge pricing, applied server-side on every ride request.
Data persistence	JSON file datastore (backend/data/db.json) with automatic seeding of demo users on first run.
Home screen	lib/home_screen.dart is referenced from main.dart as the post-login destination, but the file itself is currently empty — the rider and driver home UI has not yet been implemented.
Client–server integration	The REST API is fully implemented and independently runnable (node server.js). The Flutter client does not yet call it — no HTTP client package is declared in pubspec.yaml, so the two layers are built but not yet wired together.